MAHARASHTRA STATE BOARD OF TECHNICAL EDUCATION, MUMBAI

A Project Report on

Text Document Categorization System using Machine Learning

Submitted as a partial fulfillment for the award of

Diploma in Computer Engineering

Submitted by

Sonawane Monishka Nitin

Torawane Mayuri Anil

Waswani Dia Dinesh

Chaudhari Yogeshwari Sudhir

DEPARTMENT OF COMPUTER ENGINEERING

S.S.V.P.S's Bapusaheb Shivajirao Deore Polytechnic, Dhule

(Institute Code : 0059)

2025 – 2026

A Project Report on

Text Document Categorization using Machine Learning

Submitted as a partial fulfillment for the award of

Diploma in Computer Engineering

Submitted by

Sr No.	Name	Enrollment Number
1	Monishka Nitin Sonawane	23610960288
2	Mayuri Anil Torawane	23610960295
3	Dia Dinesh Waswani	23610960304
4	Yogeshwari Sudhir Chaudhari	23610960305

Guided by.

Mr. C. P. Bhamare

DEPARTMENT OF COMPUTER ENGINEERING

S.S.V.P.S's Bapusaheb Shivajirao Deore Polytechnic, Dhule

(Institute Code : 0059)

2025 – 2026

MAHARASHTRA STATE BOARD OF TECHNICAL EDUCATION, MUMBAI

S.S.V.P.S's Bapusaheb Shivajirao Deore Polytechnic, Dhule

DEPARTMET OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that,

Sr. No.	Roll No.	Name	Enrollment Number	Exam Seat number
1	96	Monishka Nitin Sonawane	23610960288	405659
2	102	Mayuri Anil Torawane	23610960295	405665
3	109	Dia Dinesh Waswani	23610960304	405672
4	110	Yogeshwari Sudhir Chaudhari	23610960305	405673

has satisfactorily completed project entitled "**Text Document Categorization using Machine Learning**" in partial fulfillment for the **Diploma Course in Computer Engineering** of the **Maharashtra State Board of Technical Education** at our polytechnic during the academic year **2025-2026**. The project is completed by a group consisting of four persons under the guidance of the Faculty Guide.

Date :

Place: Dhule

Signature:

Name : Mr. C. P Bhamare

Faculty Guide

Signature:

Name :

External Examiner

Signature:

Name : Mr. Nishad Patel

H.O.D.

Signature:

Name : Prof. Sanjay Hire

Principal.

Acknowledgment

We take this opportunity to express our sincere gratitude to all those who supported and guided us throughout the completion of our project on “Text Document Categorization using Machine Learning.”

First and foremost, we express our heartfelt thanks to our respected Head of Department, Mr. N. D. Patel, for his constant encouragement and support. His guidance played a significant role in the successful completion of this project.

We would also like to express our deep gratitude to our project mentor, Mr. C. P. Bhamare**,** for his valuable guidance, continuous supervision, and insightful suggestions throughout the project. His motivation and support greatly contributed to shaping this work.

We are also thankful to the various research papers, online resources, and datasets that provided us with essential knowledge of machine learning and text classification techniques, which were crucial for developing this system.

We extend our sincere thanks to all the faculty members of our department for their support and encouragement during the development of this project.

Your sincerely,

Sonawane Monishka Nitin

Torawane Mayuri Anil

Waswani Dia Dinesh

Chaudhari Yogeshwari Sudhir

Abstract

In the current digital era, a massive amount of online news content is generated every day across various domains such as politics, sports, entertainment, technology, and more. This rapid increase in textual data creates significant challenges in terms of organization, management, and efficient retrieval of relevant information. Since most of this data is unstructured and scattered across different platforms, users often find it difficult to search for important news quickly, and there is also a risk of data redundancy and loss of valuable information. To address this problem, this project proposes a machine learning-based approach for automatic text document categorization. The system utilizes two supervised learning algorithms, namely Random Forest and Decision Tree, to classify news articles into predefined categories based on their content. A real-world dataset is collected from Kaggle, which is used for training and testing the models. The project involves several stages including data preprocessing, text cleaning, feature extraction, model training, and evaluation. By analyzing patterns and relationships within the textual data, the algorithms are able to accurately group similar news articles together. Furthermore, a comparative analysis is performed to evaluate the performance of both algorithms in terms of accuracy and efficiency, helping to determine the most suitable model for this task. The developed system not only improves the organization of large-scale news data but also enhances the user experience by enabling faster and more accurate information retrieval. This project can be further extended for advanced applications such as personalized news recommendation, content filtering, and trend analysis, making it highly useful in modern information systems.

Content Page

Chapter Number	Title	Page Number
	Abstract	i
	List of Tables	ii
	List of Figures	iii
1	Introduction or Background of Industry	1-2
2	Literature survey and Specification	3-4
2.1	Study of existing system / Review of research paper	3
2.2	Limitations of existing system / Problems discussed in research papers	3
2.3	Problem definition	3
2.4	Specification	4
3	Scope of Project	5-6
3.1	Functional Scope	5
3.2	Limitations	6
4	Methodology with action plan	7-12
4.1	Overall Working of System	7
4.2	Flow of Operations	7
4.3	Block Diagram of System	8
4.4	Action Plan	10
4.5	Tools and Technologies	12

Chapter Number	Title	Page Number
5	Detail of Design, Working and Processes	13-18
6	Result and Application	19-30
7	Conclusion and Future Scope	31-32
8	Testing	33-35
9	References and Bibliography	34

List of Figures

Sr. No.	Title of Figure	Page No.
4.3	Block Diagram of System	8
5.1	ML-based Text Document Categorization System working Flowchart	13
5.2	Training Phase Flowchart	15
5.3	Naïve Bayes Model Working Diagram	17
6.1.1	Landing Page	19
6.1.2	Zip File Upload & Processing Pipeline	20
6.1.3	Processing Metrics	22
6.1.4	Document Categories	23
6.1.5	Django Administration	24
6.1.6	Document Management and Categorization Interface in Django Admin	25
6.1.7	Document Detail View in Django Admin	26
6.1.8	User Interface for Document Classification Result	27
6.1.9	Politics Folder – Categorized Document View	28
6.1.10	System Workflow	29

List of Tables

Sr. No.	Title of Table	Page No.
1	Processing Pipeline Steps	21
2	Test Cases for file upload module	36
3	Defect Report 1	37
4	Defect Report 2	39
5	Defect Report 3	41

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

News text is very much essential for having timely information which is also required in future. Now a days, Internet is full of news content, and it became a challenging task to store this information efficiently so that it can be extracted easily in future on time. News text also contain so much of irrelevant information which one is of no use and user want to read only crisp point about the news article. For this purpose, news text classification concept is used where text is classified based on certain attributes like topic, language, author name, date etc. (Stein, 2020). The major application of this classification is that it becomes easy to search a particular category of news for which a user may be interested which saves time, effort, and money. Supervised machine learning algorithms can be used for this purpose where a model is trained using training data and results are obtained for a particular dataset.

1.2 THE APPLICATION DOMAIN

News content is one of the most important forms of digital data and requires effective categorization due to the large amount of information published daily. News categorization involves organizing news articles into predefined groups such as Politics, Sports, Business, Technology, and Entertainment. This structured approach helps in managing news content efficiently.

By categorizing news articles, the website enables users to quickly access news that matches their interests without spending excessive time searching through irrelevant content. This method reduces information overload and allows users to stay updated on specific topics more efficiently. As a result, news categorization significantly improves user experience by offering faster navigation, better content discovery, and more personalised access to digital news information.

1.3 THE NEED FOR THE PROJECT

In today's digital world, users are exposed to a continuous stream of news from multiple online sources. Due to this overwhelming amount of information, users often find it difficult to identify news that is relevant to their interests. An efficient news categorization system is required to address this challenge.

This project is needed to automatically organize news articles into well-defined categories such as Politics, Sports, Business, Technology, and Entertainment. By doing so, the system simplifies content browsing and enables users to focus on topics they care about the most. The project enhances usability, saves users' time, and ensures structured presentation of news content, making digital news platforms more user-friendly and effective.

1.4 BACKGROUND OF INDUSTRY

The rapid growth of digital technology has led to an exponential increase in the generation of textual data from various sources such as news articles, emails, social media, research papers, and online platforms. Managing and organizing this large volume of unstructured text data has become a major challenge for organizations and individuals.

The field of Artificial Intelligence (AI)**,** particularly Machine Learning (ML) and Natural Language Processing (NLP)**,** plays a crucial role in addressing this challenge. Text document categorization is an important application of supervised machine learning, where documents are automatically classified into predefined categories based on their content.

Traditionally, document classification was performed manually, which is time-consuming, error-prone, and inefficient when dealing with large datasets. With advancements in machine learning algorithms and computational power, automated text classification systems have become more accurate and efficient.

Various techniques such as text preprocessing, feature extraction methods like TF-IDF, and classification algorithms like Naïve Bayes, Support Vector Machines, and Deep Learning models are widely used in this domain. Among these, Naïve Bayes is particularly popular due to its simplicity, speed, and effectiveness in handling high-dimensional text data.

Industries such as news agencies, email service providers, digital libraries, and content management systems extensively use text categorization for applications like spam detection, sentiment analysis, topic classification, and information retrieval.

Despite these advancements, challenges such as handling large-scale data, improving accuracy, and adapting to dynamic content still exist. Therefore, developing efficient and scalable text categorization systems remains an important area of research and application

. CHAPTER 2

LITERATURE SURVEY AND SPECIFICATION

2.1 STUDY OF EXISTING SYSTEM / MANUAL PROCESS

In the existing system, news articles are published in large numbers across digital platforms without proper automated organization. Users often rely on manual searching or scrolling through multiple news items to find information related to their interests. In many cases, news categorization is done manually by editors, which is time-consuming and inefficient.

2.2 LIMITATIONS OF EXISTING SYSTEM / PROBLEMS DISCUSSED IN RESEARCH PAPERS

The existing approaches to text document categorization primarily rely on manual or semi-automated methods, which present several limitations. One of the major challenges is information overload, where the vast amount of unstructured textual data makes it difficult for users to quickly identify and access relevant information

Manual categorization of documents is time-consuming and labor- intensive, requiring continuous human involvement. This process is also prone to errors and inconsistencies, as different individuals may classify the same content differently based on their understanding.

Furthermore, traditional methods lack scalability, making them inefficient when handling large volumes of data that are continuously generated from digital platforms. As the amount of textual content grows, maintaining accuracy and consistency becomes increasingly difficult.

2.3 PROBLEM DEFINITION

The system is a web-based news classification application that automatically cat-egorizes news articles into areas such as Politics, Sports, Business, Technology, and Entertainment. It eliminates manual sorting, which is slow due to the large volume of news data. The system uses machine learning algorithms like Naive Bayes, SVM, and Logistic Regression to analyze text and predict the correct category. It is trained on the 20 Newsgroups dataset to ensure fast and accurate classification through an easy-to-use web interface.

2.4 SPECIFICATION

The system is an “Text document categorization using Machine Learning” de-signed to automatically classify textual data into predefined categories using supervised machine learning techniques. The system will allow users to input text documents, which will then be pro-cessed and analyze determine their appropriate category based on learned patterns. The system in-cludes modules for data preprocessing, feature extraction, model training, and classification, ensur-ing efficient handling of unstructured text data. It is designed with a simple and user-friendly inter-face that enables users to easily upload or enter text and obtain accurate classification results. The system aims to reduce manual effort, improve classification accuracy, and provide faster information retrieval. By utilizing intelligent algorithms and structured processing techniques, the system en-hances the organization of large volumes of textual data and supports applications such as news categorization, email filtering, and content management systems.

CHAPTER 3
SCOPE OF PROJECT

3.1 FUNCTIONAL SCOPE

The project focuses on developing a news categorization website that organizes news articles into predefined categories such as Politics, Sports, Business, Technology, and Entertainment. The scope of the Text Document Categorization Using Machine Learning lies in the field of specifically in Natural Language Processing (NLP) and supervised machine learning. The system is designed to automatically classify textual data into predefined categories, enabling effi-cient organization and management of large volumes of unstructured text. It supports document-level classification and can be applied to various types of textual data such as news articles, emails, and digital documents. The system includes functionalities such as text preprocessing, feature extraction, model training, and automated classification, which help in transforming raw text into structured information. This improves information retrieval, reduces manual effort, and enhances decision-making processes. The scope of text categorization extends across multiple domains, including spam detection, sentiment analysis, topic labeling, and content filtering, making it a widely applicable solution in modern digital systems.

- **Automatic Document Classification**

This system automatically categorizes documents into predefined classes (such as technical, legal, financial, etc.) without human intervention. It reduces manual effort and increases accuracy by analyzing the content of documents and assigning them to the most relevant category.

- **Machine Learning-Based Prediction**

The project uses machine learning algorithms to predict the category of a document. The model learns patterns from training data and then applies this knowledge to classify new, unseen documents accurately.

- **Dataset-Based Model Training**

A dataset containing labelled documents is used to train the model. During training, the system learns relationships between text features and their corresponding categories, improving its prediction capability over time.

- **Web-Based Application**

The system is implemented as a web application, allowing users to upload documents, view classification results, and interact with the system through a user-friendly interface accessible via a browser.

- **Document Management and Visualization**

The application provides features to manage uploaded documents efficiently. It may include options to view, search, filter, and visualize categorized documents using charts or dashboards for better understanding.

- **Efficient Information Organization**

By automatically categorizing documents, the system helps organize large volumes of data in a structured way. This improves retrieval speed, reduces confusion, and enhances overall productivity.

3.2 LIMITATIONS

The project is limited to categorising news articles only into predefined categories. It does not include advanced personalisation features such as user profiles or recommendation systems. Real- time news updates, multilingual support, and sentiment analysis are not covered in this project. The system also does not verify the authenticity or credibility of news

CHAPTER 4
METHODOLOGY

4.1 OVERALL WORKING OF THE SYSTEM

The system is an automated news categorization system that classifies news articles into predefined categories such as Politics, Sports, Business, Technology, and Entertainment using supervised machine learning techniques like Naive Bayes, SVM, and Logistic Regression. The system takes raw news text as input, processes it to remove noise, extracts important features, and applies a trained machine learning model to predict the appropriate category. Once categorized, the news articles are displayed on the website under their respective categories, enabling users to easily access relevant news content.

4.2 FLOW OF OPERATIONS

- **Input (Text):**

The system receives raw text data such as a news article or document.

- **Feature Extraction:**

The input text is transformed into a feature set in the form of a numerical vector using techniques like TF-IDF or Bag of Words. This step converts unstructured text into structured data.

- **Feature Set:**

The extracted features represent the important terms and patterns present in the text and are used as input for the classification model.

- **Text Classification Model:**

The feature vectors are fed into a trained text classification model (for example, Naive Bayes), which analyses the data based on learned patterns.

- **Predicted Tags:**

The model produces predicted tags or categories for the input text, indicating the class to which the document belongs

4.3 BLOCK DIAGRAM OF SYSTEM

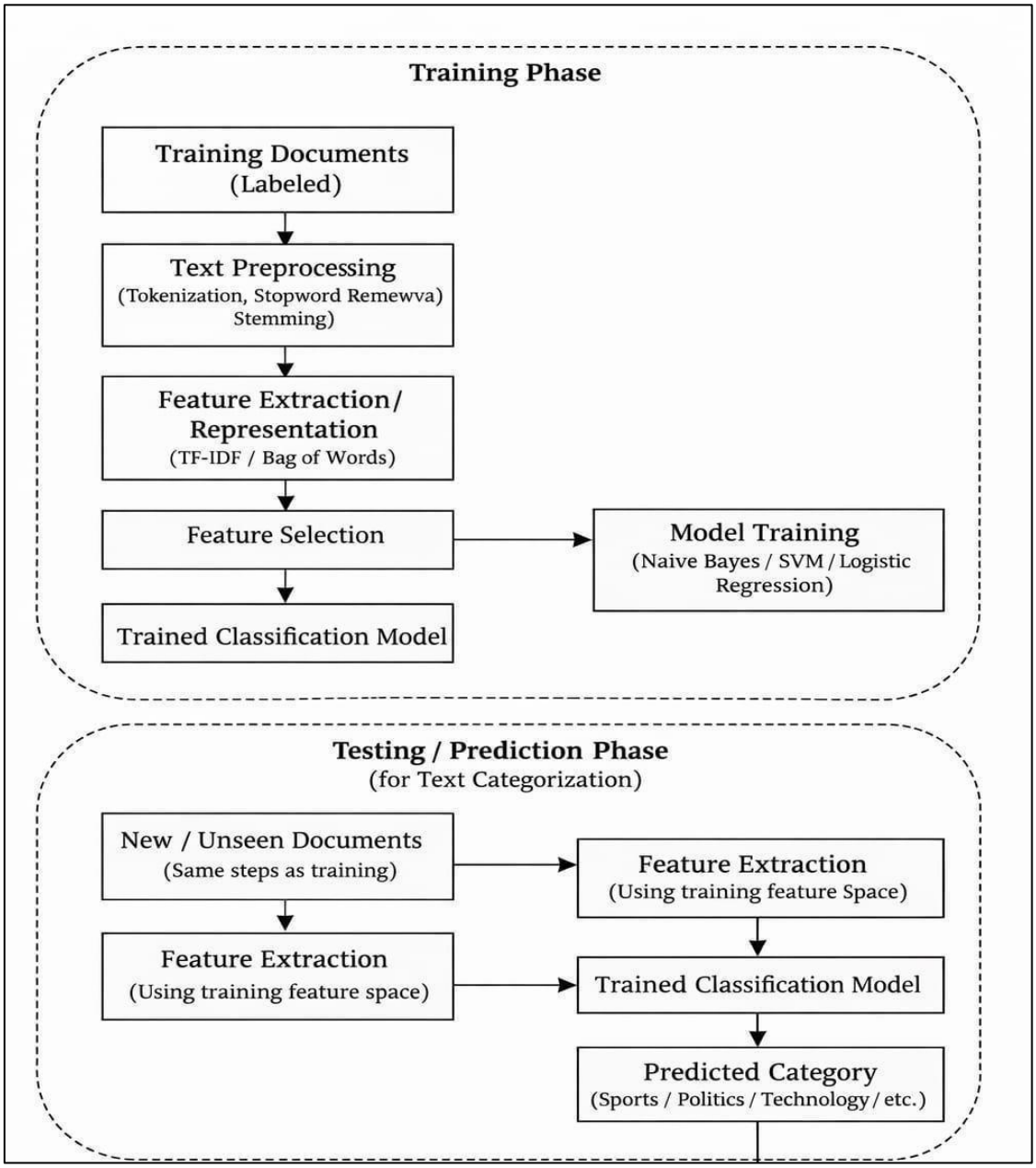


Fig 4.3 System Flow

This architecture consists of two main phases: the Training Phase and the Testing/Prediction Phase.

4.3.1 Training Phase:

- **Training Documents (Labeled):**

The process starts with a set of labeled training documents, meaning news articles that are already categorized into known classes (e.g., Sports, Politics, Technology).

- **Text Preprocessing:**

The raw text undergoes preprocessing steps to clean and prepare the data for analysis. This typically includes:

- **Tokenization:** Splitting text into words or tokens.
- **Stop word Removal:**

Removing common words (like "the", "is") that do not add meaningful information.

- **Stemming:** Reducing words to their base or root form (e.g., "running" to "run").
- **Feature Extraction/Representation:**

The pre-processed text is converted into numerical features that machine learning algorithms can work with. Common methods include:

- **TF-IDF (Term Frequency-Inverse Document Frequency):**

Weighs the importance of words in the documents.

- **Bag of Words:** Represents text as the frequency of each word.
- **Feature Selection:**

Fromthe extracted features, the most relevant ones are selected to reduce dimensionality and improve model performance.

- **Model Training:**

The selected features are used to train classification models using machine learning algorithms such as: Naive Bayes, Support Vector Machine (SVM), Logistic Regression

- **Trained Classification Model:**

The output of this phase is a trained model that can categorize new documents based on learned patterns.

4.3.2 Testing / Prediction Phase (For Text Categorization):

- **New / Unseen Documents:**

When new news articles arrive that need to be categorized, they go through the same preprocessing steps as the training data.

- **Feature Extraction:**

Features are extracted from the new documents using the same feature space learned during training, ensuring consistency.

- **Trained Classification Model:**

The trained model from the training phase is then used to analyze the features of the new documents.

- **Predicted Category:**

The system outputs the predicted category for each new document, such as Sports, Politics etc.

- **Trained Classification Model:**

The trained model from the training phase is then used to analyze the features of the new documents.

- **Predicted Category:**

The system outputs the predicted category for each new document, such as Sports, Politics etc.

4.4 ACTION PLAN

The action plan describes the systematic steps followed for the successful development of the Text Document Categorization using Machine Learning.

1. Problem Identification

The problem of handling and organizing large volumes of unstructured text data was identified. The need for an automated and accurate document categorization system was recognized.

2. Requirement Analysis

System requirements were analyzed, including user input (text/ZIP file), preprocessing, model training, and automatic categorization into predefined classes.

3. Literature Survey

Relevant research papers and existing text classification systems were studied to understand machine learning techniques such as TF-IDF and Naive Bayes, along with their advantages and limitations.

4. Dataset Selection and Preparation

The 20 Newsgroups dataset was selected for training and testing. The dataset contains multiple categories, making it suitable for building a classification model.

5. Data Preprocessing

The collected text data was cleaned and prepared using techniques such as tokenization, stop-word removal, and text normalization to improve data quality.

6. Feature Extraction (TF-IDF)

The preprocessed text was converted into numerical form using TF-IDF (Term Frequency–Inverse Document Frequency) to make it suitable for machine learning algorithms.

7. Model Training

A supervised machine learning model using the Naive Bayes algorithm was trained on the processed dataset to learn patterns and relationships between text and categories.

8. Model Evaluation

The trained model was evaluated using test data to check its accuracy and performance in classifying documents correctly.

9. Prediction Module Development

A prediction system was developed where new input text or documents are analyzed and classified into one of the predefined categories based on the trained model.

10. ZIP File Handling Implementation

A feature was implemented to allow users to upload ZIP files containing multiple text documents. Each document is extracted and processed individually for categorization.

11. Category Assignment Logic

The system assigns documents to one of the eight predefined categories. If a document does not match any category with sufficient confidence, it is placed in the "Other" category.

12. System Integration

All modules, including preprocessing, feature extraction, model training, and prediction, were integrated into a single working system.

13. Testing and Validation

The complete system was tested using different input samples to ensure accuracy, efficiency, and proper functioning of all modules.

14. Result Analysis and Optimization

System results were analyzed, and improvements were made to enhance classification accuracy and overall performance.

15. Documentation and Presentation

The complete project report was prepared, and a presentation was created to demonstrate the working and outcomes of the system.

16. Final Submission

The project was reviewed, corrected based on feedback, and submitted along with all necessary documentation.

4.5 TOOLS & TECHNOLOGIES

- Programming Language: Python
- Frontend: HTML/CSS.
- Backend: Python Django Framework
- Database: SQLite
- Development Tools: Visual Studio Code.
- Dataset: 20 News Group.
- Operating System: Windows Operating System(Version 11).

CHAPTER 5

DETAILS OF DESIGNS, WORKING AND PROCESSES

5.1 WORKING OF SYSTEMS

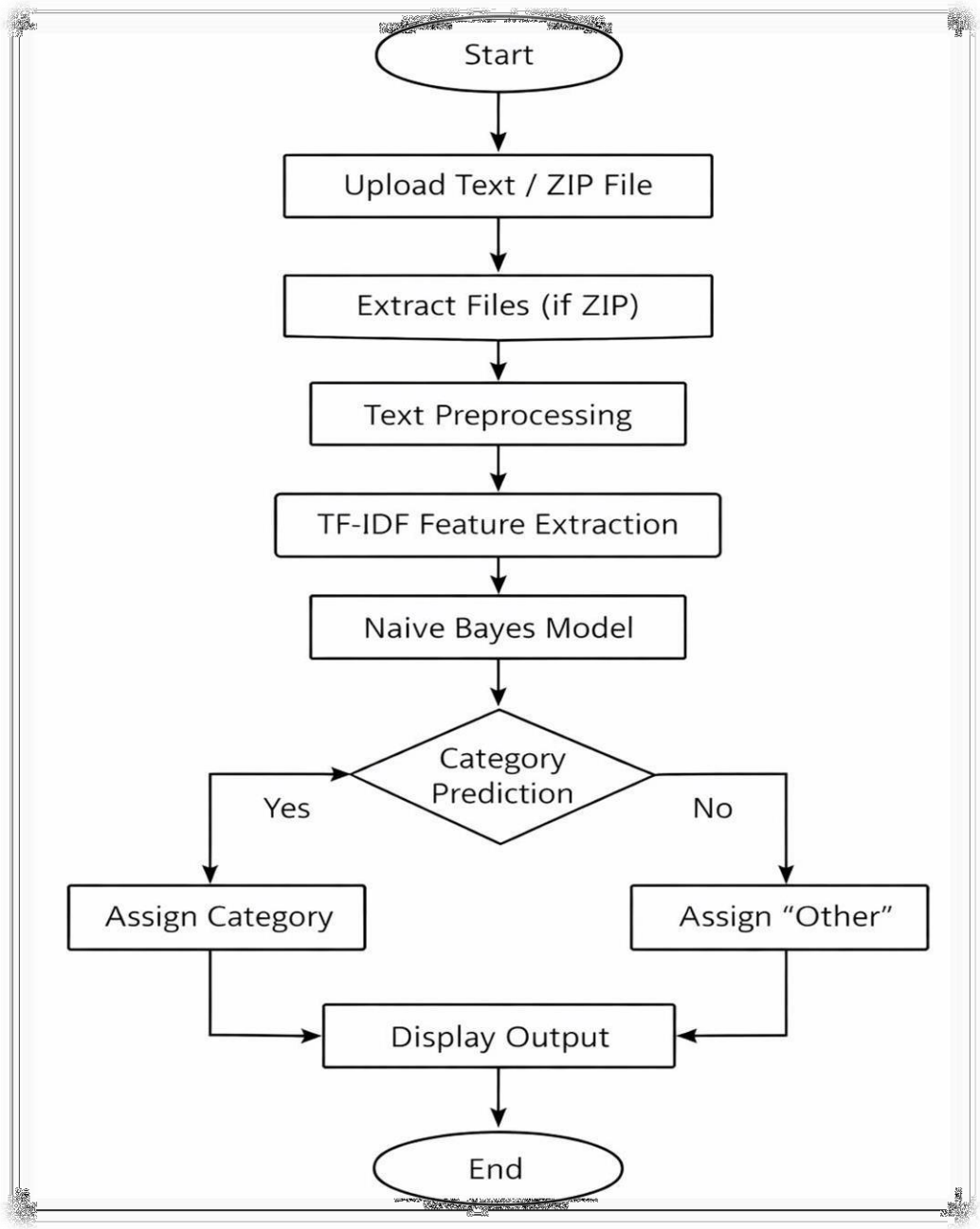


Fig 5.1 Text Document Categorization System using Machine Learning working Flowchart

Figure 5.1 represents the complete workflow of the Text Document Categorization using Machine Learning, showing how input data is processed and converted into categorized output using machine learning techniques.

The process begins with the Start point, where the user initiates the system. The user then uploads input data in the form of a text file or a ZIP file containing multiple documents. If a ZIP file is uploaded, the system extracts all the files and processes them individually.

After input collection, the system performs text preprocessing, which is a crucial step for improving data quality. In this stage, unwanted elements such as punctuation, special characters, and stop words are removed. The text is also tokenized and normalized to ensure consistency and better analysis.

Once preprocessing is completed, the system applies TF-IDF (Term Frequency–Inverse Document Frequency) for feature extraction. This technique converts textual data into numerical vectors by identifying the importance of words in each document. It helps the model understand which words are significant for classification.

The processed data is then passed to the Naive Bayes classification model, which has been previously trained using the 20 Newsgroups dataset. The model analyzes the input features and predicts the most appropriate category for each document based on learned patterns.

After prediction, a decision step is performed to check whether the classification confidence is sufficient. If the system finds a strong match, the document is assigned to one of the predefined categories. If the confidence level is low or no suitable category is found, the document is assigned to the "Other" category.

Finally, the categorized results are displayed to the user through the output interface. The system ensures that all documents are processed and categorized efficiently. The workflow ends at the End point, indicating successful completion of the process.

This automated workflow reduces manual effort, improves accuracy, and provides a fast and efficient solution for text document categorization

5.2 TRAINING PHASE FLOWCHART

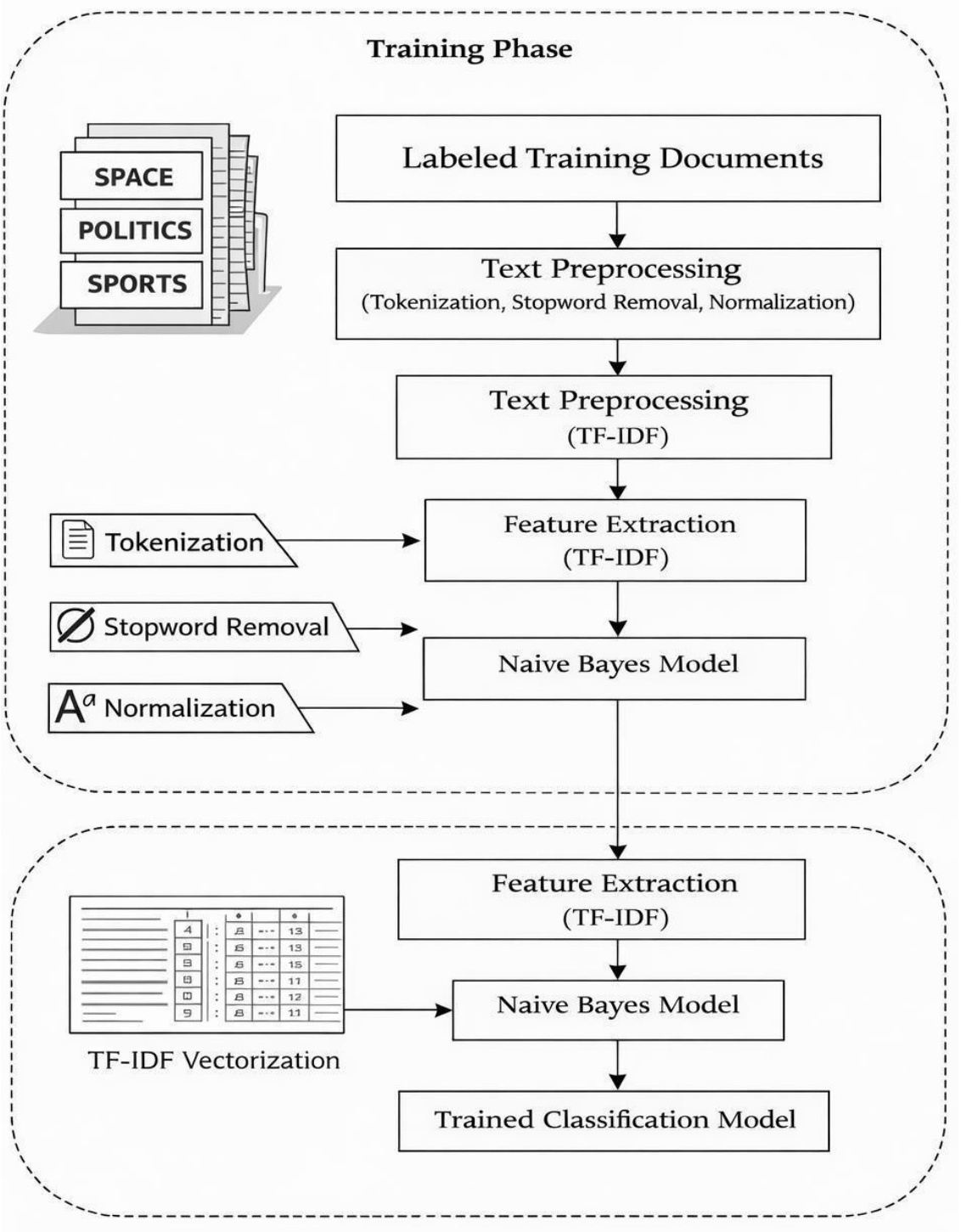


Fig 5.2 Training Phase Flowchart

The diagram represents the complete training process of the Text Document Categorization using Mahine Learning. This phase is responsible for building a machine learning model using the

20 Newsgroups dataset so that it can classify new documents accurately.

The process begins with labelled training documents, where each document is assigned to a specific category such as sports, politics, or technology. These datasets help the model learn the relationship between text content and its category.

Next, the system performs text preprocessing to improve data quality. In this stage, unwanted elements such as punctuation, special characters, and stop words are removed. The text is also tokenized and normalized to maintain consistency.

After preprocessing, the system moves to feature extraction, where text is converted into numerical form using the TF-IDF technique. This method highlights important words based on their relevance. Feature selection may also be applied to reduce dimensionality and improve performance.

The processed data is then passed to the Naive Bayes model for training. The model learns by calculating probabilities of words belonging to different categories and builds relationships between features and classes.

Finally, a trained classification model is generated, which is used to classify new or unseen documents. This phase is important as it directly affects the accuracy and performance of the system.

5.2.1 20 NEWSGROUPS DATASET

The 20 Newsgroups dataset is a widely used benchmark dataset in the field of Natural Language Processing (NLP) and machine learning for text classification tasks. It consists of approximately 18,000 to 20,000 newsgroup documents collected from 20 different categories.

Each document in the dataset represents a post or message taken from online discussion forums known as newsgroups. These documents are already labeled with their respective categories, making the dataset suitable for supervised learning.

The dataset is divided into 20 different classes, which are grouped into major categories such as Computers, Science, Recreation, Politics, Religion, and Miscellaneous. Some examples of categories include comp.graphics, sci.space, rec.sport.baseball, talk.politics.guns, and alt.atheism.

Each category contains a large number of text documents, which helps the model learn patterns and relationships between words and their respective classes. The dataset includes both training and testing data, allowing proper evaluation of machine learning models.

The 20 Newsgroups dataset is widely used because it provides real-world text data with diverse topics, making it ideal for building and testing text classification systems. It is especially useful for

evaluating algorithms like Naive Bayes, Support Vector Machines, and other machine learning models.

In this project, the dataset is used to train the classification model so that it can accurately categorize new and unseen text documents into predefined category.

5.3 NAÏVE BAYES MODEL WORKING

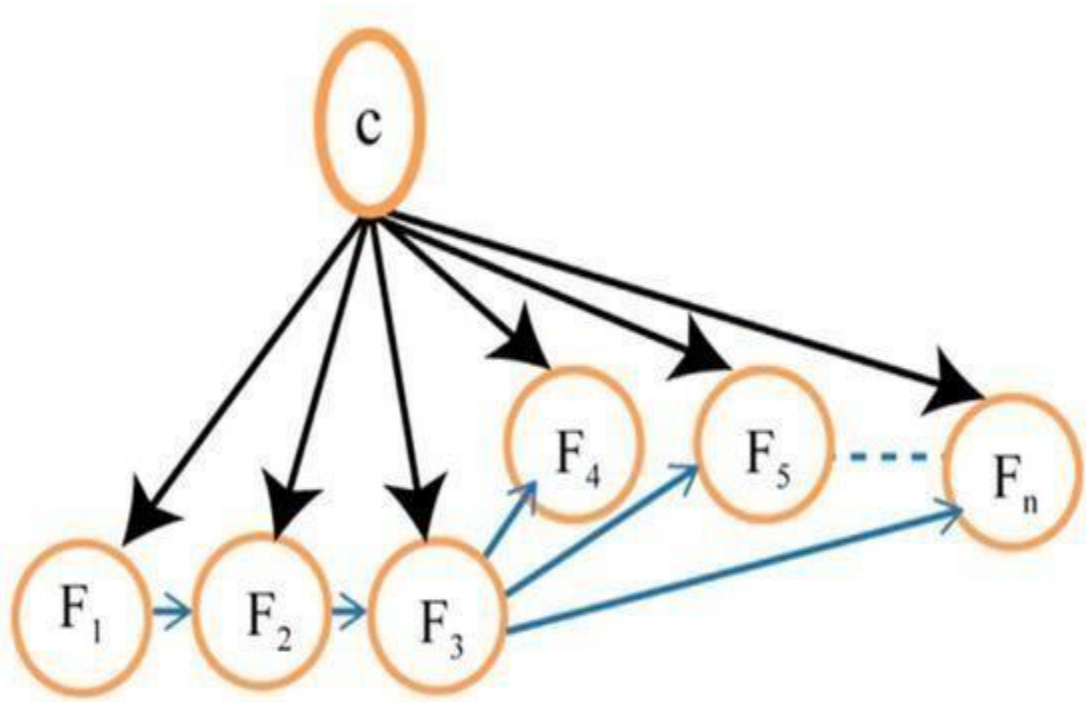


Fig 5.3 Naïve Bayes Model Working Diagram

Naive Bayes: Naive Bayes is frequently used in TC due to its simplicity, efficiency, and resilience, especially in document categorization and spam filtering. However, while the assumption of feature independence simplifies calculation, it can reduce efficiency when features are highly linked. Figure 5.2.1 shows how Naïve Bayes works.

Nodes:

- Topmost node "C" represents the class label of the text document.
- The nodes labeled F_1, F_2, …, F_n represent features (words, phrases, or attributes) extracted from the text.

Arrows:

- The black arrows from $C \rightarrow \{F_1, F_2, \ldots, F_n\}$ indicate that the classification decision directly influences the features. This suggests a Naïve Bayes model assumption, where features are

conditionally independent given the class.

- The blue arrows between features represent feature dependencies or correlations (e.g., word co-occurrence relationships). This indicates that some features depend on each other, making the model more complex than Naïve Baye

CHAPTER 6

RESULT AND APPLICATION

6.1 RESULT:

6.1.1 Landing Page

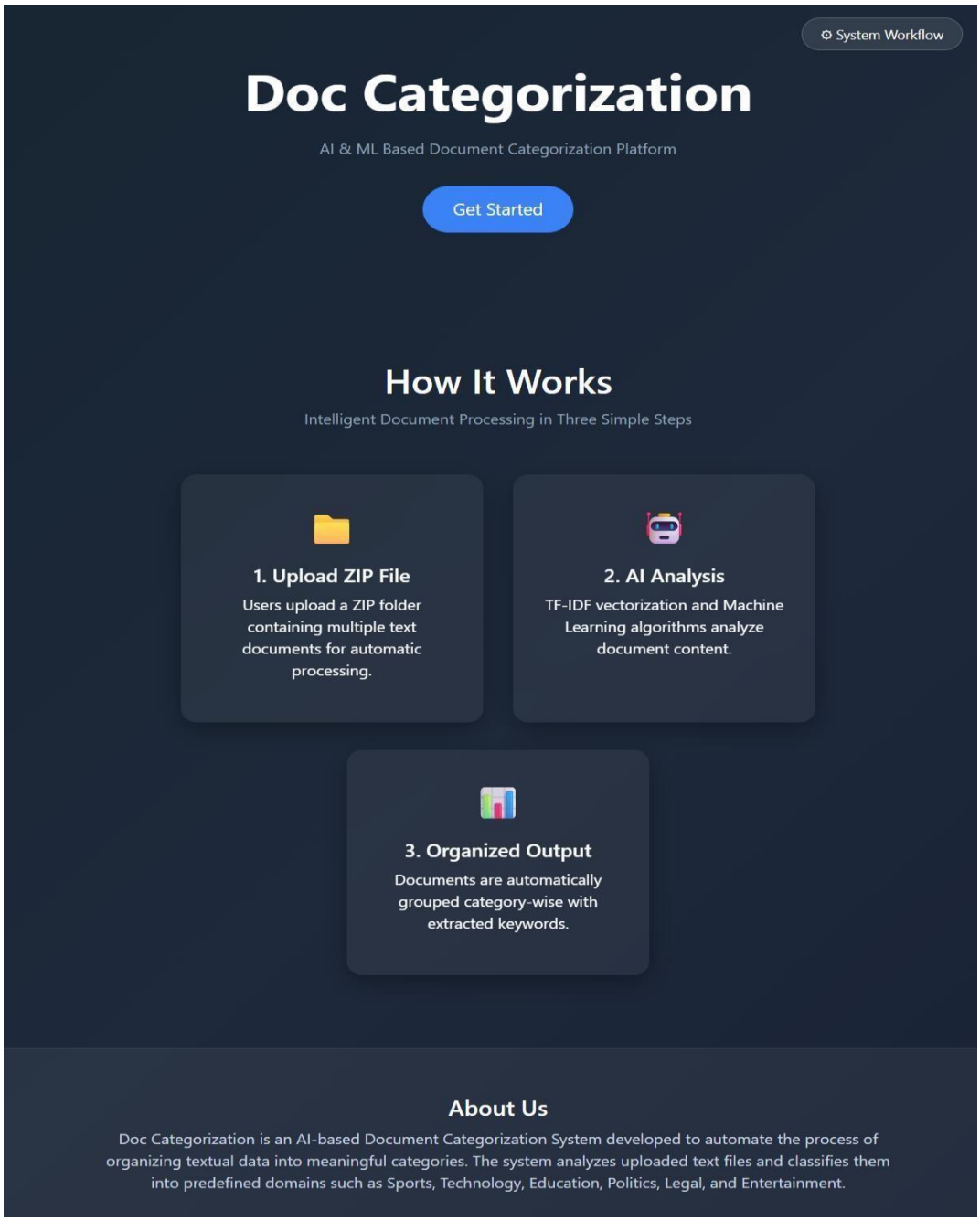


Fig 6.1.1 Landing Page

The landing page serves as the initial interface of the Text Document Categorization using Machine Learning. It introduces the purpose of the system and provides users with a clear understanding of how the application works. The design follows a clean and structured layout, ensuring ease of use and better user interaction.

System Interaction Details:

The interface highlights the concept of automatic document classification using machine learning techniques and guides users through the basic workflow before they begin using the system.

- At the top of the page, the system title "Doc Categorization" is displayed along with a short tagline indicating that it is an Machine Learning-based platform, helping users understand the purpose of the application.
- A "Get Started" button is provided to allow users to proceed to the document processing stage.
- Below this, a "How It Works" section explains the workflow in three steps: uploading a ZIP file, performing ML-based analysis using TF-IDF and machine learning, and generating categorized output.
- Each step is presented in a card layout with icons for better understanding.
- At the bottom, an "About Us" section briefly describes the system and its ability to automatically categorize documents into predefined domains.

6.1.2 ZIP FILE UPLOAD AND PROCESSING PIPELINE

Fig 6.1.2 Zip File Upload & Processing Pipeline

This page represents the main working interface of the Text Document Categorization using Machine Learning, where users upload documents and the system processes them step by step.

The user selects a ZIP file and clicks on the “Upload” button to start the process. A processing pipeline is displayed, where each stage is highlighted as it gets completed, allowing users to track progress in real time.

The process begins with File Upload, followed by Text Preprocessing, where unwanted elements are removed. Next, Feature Extraction is performed using techniques like Bag of Words and TF-IDF.

After this, the Model Processing stage is activated, where the Naive Bayes model analyzes the data and predicts categories. Finally, the Result Generation stage is highlighted, showing that the documents have been successfully categorized.

This step-by-step highlighting improves transparency and helps users understand the system workflow easily.

Table 1 Processing Pipeline Steps

Sr. No.	Pipeline Steps	Description
1	File Upload	User selects and uploads a ZIP file containing text documents
2	Upload Confirmation	System verifies and highlights successful file upload
3	Text Preprocessing	Cleaning of text (removal of stop words, punctuation, normalization)
4	Feature Extraction	Conversion of text into numerical form using BOW and TF-IDF
5	Model processing	Naive Bayes model analyzes data and predicts categories
6	Result Generation	Final categorized output is prepared and displayed

6.1.3 PROCESSING METRIC

Fig 6.1.3 Processing Metrics

This page displays the performance and evaluation details of the Text Document Categorization using Machine Learning after processing the uploaded documents.

- Total Files Processed**:** Shows the total number of documents that have been successfully processed by the system from the uploaded ZIP file.
- Average Confidence: Indicates the average confidence score of the model while predicting categories. It represents how confident the model is about its predictions.
- Training Accuracy: Displays the accuracy of the model during the training phase. It reflects how well the model has learned from the training dataset.
- Precision: Represents the correctness of the model's predictions. A higher precision means the model is making more accurate category assignments.
- Progress Bars Visualization: Each metric is visually represented using progress bars, making

it easier for users to understand performance levels at a glance.

- Model Performance Section: This section provides an overview of overall model metrics and may include graphical representation for better analysis.
- Navigation Option: A “Go to Dashboard” button is provided to allow users to move to the main dashboard for further analysis or actions.

6.1.4 DOCUMENT CATEGORIES

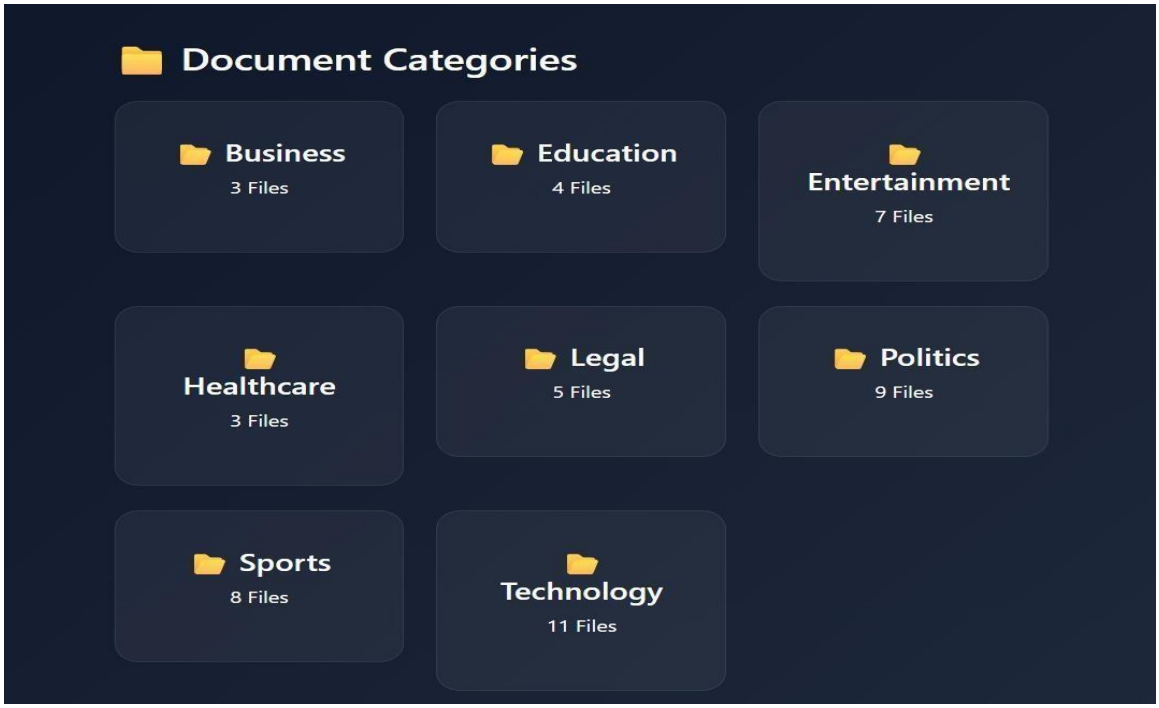


Fig 6.1.4 Document Categories

This page displays the final output of the Text Document Categorization using Machine Learning, where all processed documents are grouped into predefined categories.

Each category, such as Business, Education, Entertainment, Healthcare, Legal, Politics, Sports, and Technology, is shown in a card format along with the number of files assigned to it.

The system automatically classifies the uploaded documents and organizes them into these categories, making it easy for users to view and access categorized data.

This structured representation helps in quick understanding and efficient management of documents.

6.1.5 DJANGO ADMINISTRATION

Fig 6.1.5 Django Administration

The above image shows the Django Administration interface used in the Text Document Categorization using Machine Learning. This panel provides a centralized backend interface for managing users, permissions, and document data.

At the top of the interface, the system displays the logged-in administrator along with options such as view site, change password, and logout, ensuring secure access control.

The Authentication and Authorization section includes:

Users: Allows the admin to add, update, and manage user accounts who interact with the system.

Groups: Enables role-based access control by assigning permissions to different user groups.

The Core section contains:

Documents: This module is responsible for managing the text documents used in the system. The administrator can add new documents or modify existing ones, which are further processed by the machine learning model for categorization.

On the right-hand side, the Recent Actions panel displays the latest activities performed by the administrator, helping in tracking system operations.

Overall, this interface plays a crucial role in the project by providing an easy-to-use backend for data management, user control, and system monitoring, which supports the efficient functioning of the text document categorization process.

6.1.6 DOCUMENT MANAGEMENT AND CATEGORIZATION INTERFACE IN DJANGO ADMIN

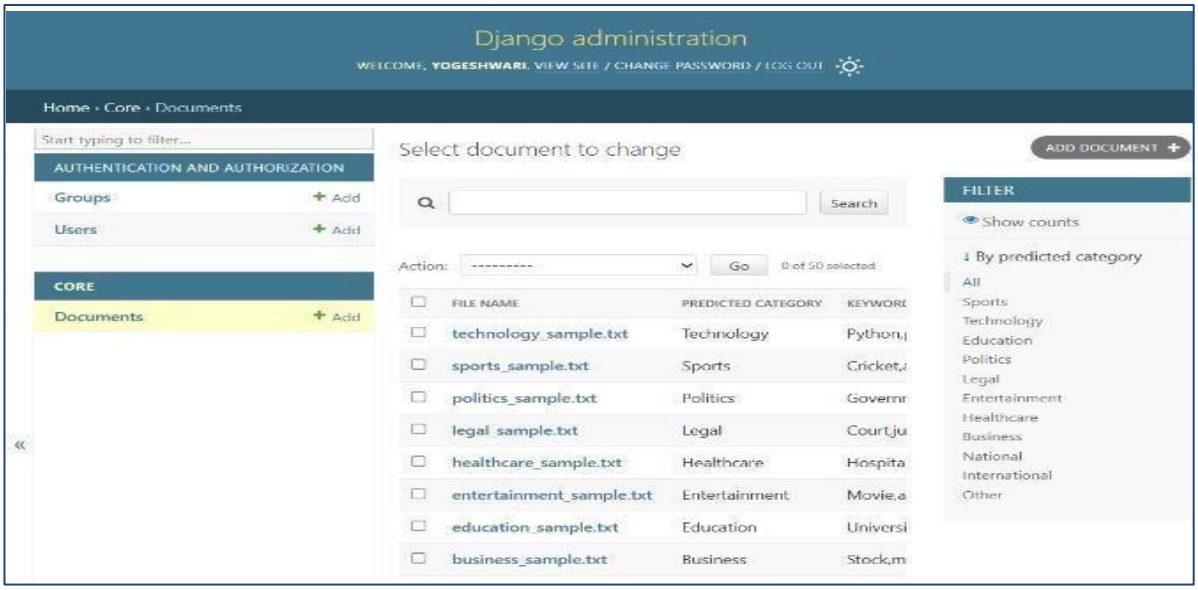


Fig 6.1.6 Document Management and Categorization Interface in Django Admin

The Fig 6.1.6 shows the Django Administration Documents module of the Text Document Categorization using Machine Learning.

This interface displays a list of text documents along with their predicted categories generated by the machine learning model, such as Technology, Sports, Politics, etc. It also shows important key-words extracted from each document.

The admin can:

- Search documents using the search bar
- Filter documents by category (right side panel)
- Add new documents using the “Add Document” button
- Perform bulk actions using checkboxes

Overall, this panel helps in managing and verifying categorized documents efficiently within the system.

6.1.7 DOCUMENT DETAIL VIEW IN DJANGO ADMIN

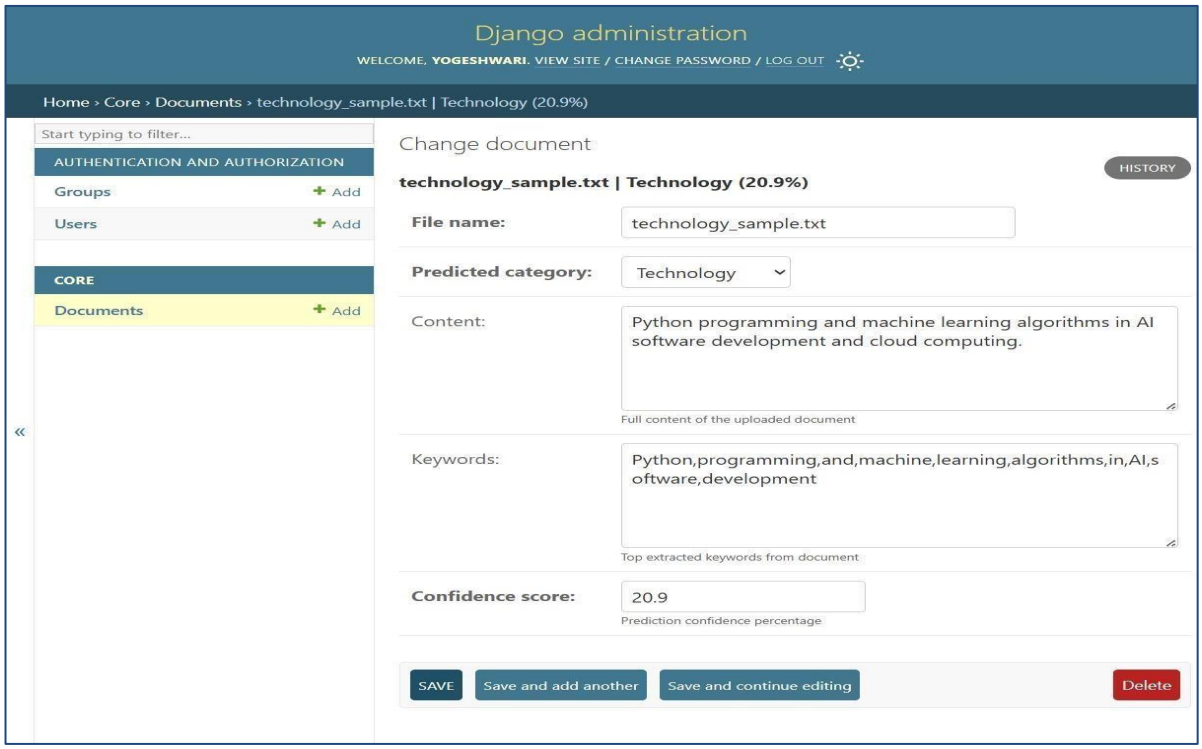


Fig 6.1.7 Document Detail View in Django Admin

The image shows the Django Administration detailed view of a single document in the Text Document Categorization using Machine Learning.

This page displays complete information about a selected document, including:

- File name of the document
- Predicted category assigned by the ML model (e.g., Technology)
- Content of the document Extracted keywords used for classification
- Confidence score indicating prediction accuracy

The admin can edit, update, or delete the document using the available options.

Overall, this interface helps in analysing and verifying individual document classification results.

6.1.8 USER INTERFACE FOR DOCUMENT CLASSIFICATION RESULT

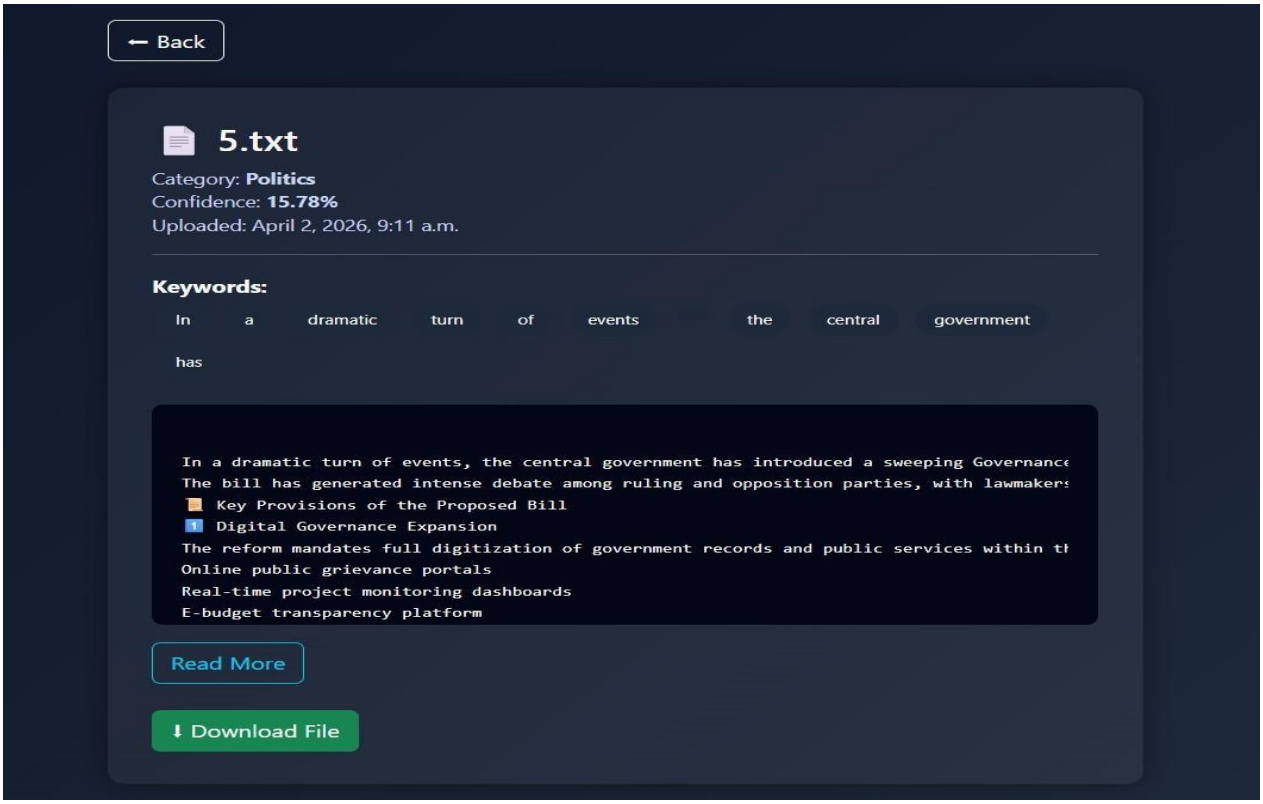


Fig 6.1.8 User Interface for Document Classification Result

The image shows the frontend interface of the Text Document Categorization using Machine Learning displaying the result of a selected document.

This screen presents:

File name (e.g., *5.txt*)

- Predicted category (Politics) assigned by the machine learning model
- Confidence score indicating prediction accuracy
- Upload date and time

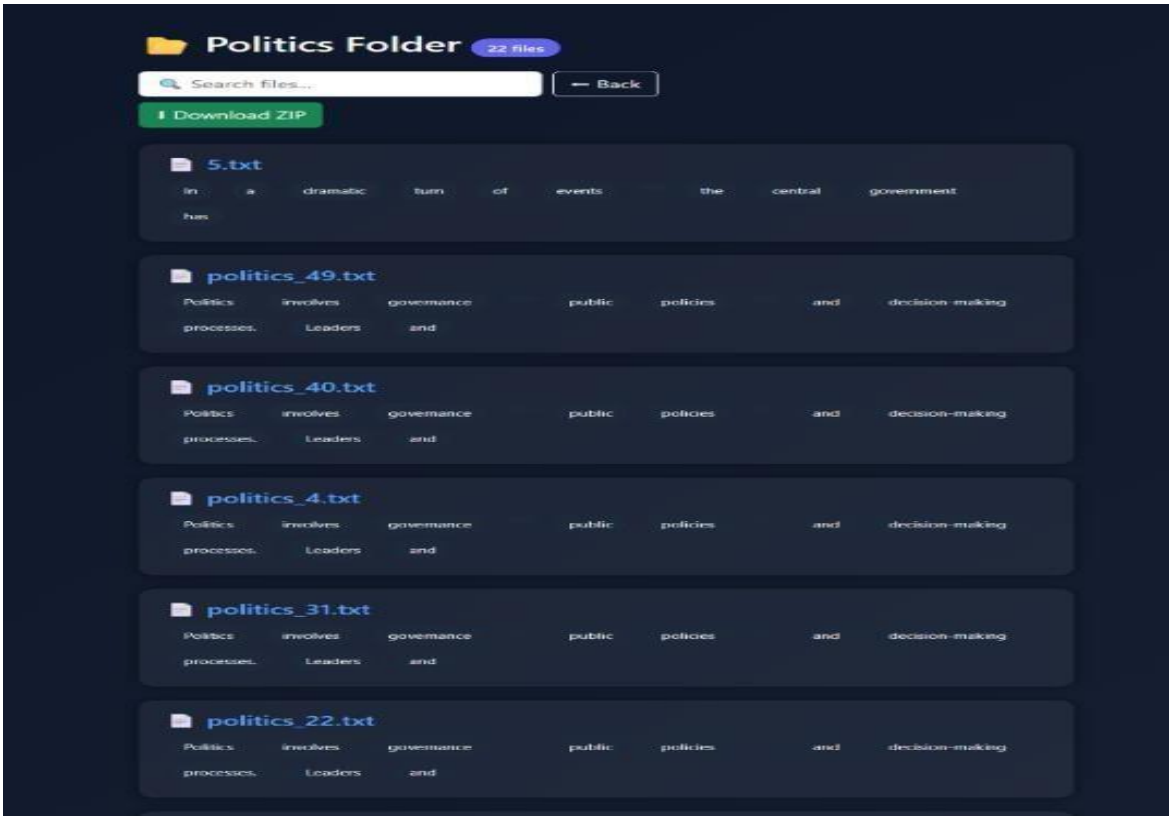
It also displays extracted keywords that help in classification and a preview of the document content.

Additional features include:

- More button to view full content
- Download File option to save the document

Overall, this interface provides users with a clear view of the classification result and document details in an interactive manner.

6.1.9 POLITICS FOLDER – CATEGORIZED DOCUMENT VIEW



6.1.9 Politics Folder – Categorized Document View

The above interface represents the categorized results page of the system after processing and classifying text documents using a Machine Learning model.

- At the top, the folder named "Politics Folder" indicates that all displayed documents belong to the politics category. It also shows the total number of files (22 files).
- A search bar is provided to allow users to quickly find specific documents within this category.
- The "Download ZIP" button enables users to download all categorized documents in compressed format for offline use.
- Each listed file (e.g., *politics_49.txt, politics_40.txt*) represents an individual text document that has been automatically classified by the system.
- Below each file name, a short preview of the document content is shown. This helps users quickly understand the context without opening the file.
- The "Back" button allows navigation to the previous page or main category selection interface.

6.1.10 SYSTEM WORKFLOW

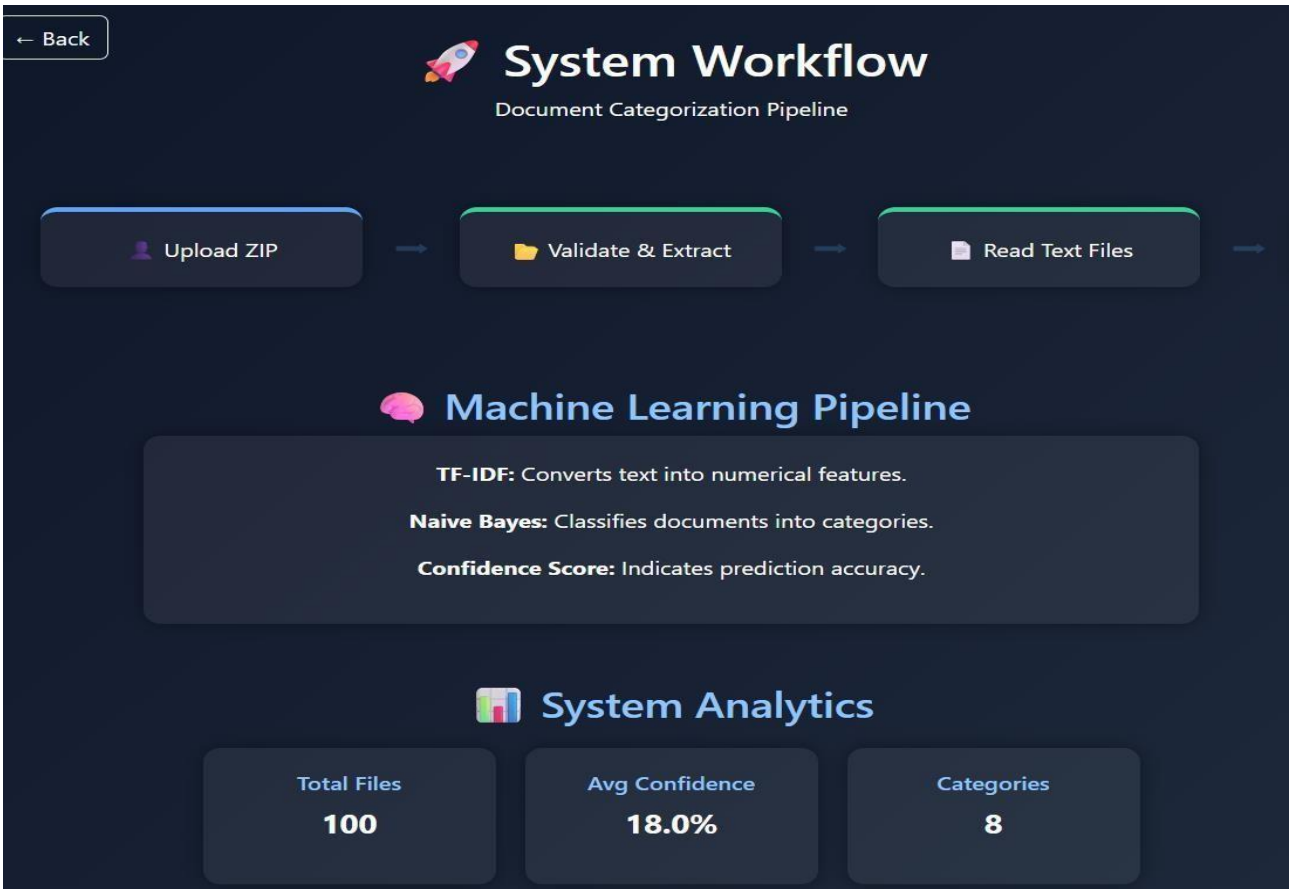


Fig 6.1.10 System Workflow

This diagram shows the system workflow of the document categorization system. It explains the step-by-step process followed by the system, starting from uploading the ZIP file, validating and extracting it, and reading the text files. After that, the machine learning pipeline processes the data using TF-IDF and Naive Bayes to classify documents. Finally, the system displays analytics like total files, confidence score, and category.

6.2 APPLICATIONS OF TEXT DOCUMENT CATEGORIZATION

Text Document Categorization plays an important role in managing and organizing large amounts of textual data across different domains. It improves efficiency, enhances user experience, and helps in extracting useful information from unstructured data.

1. Document Indexing for Information Retrieval

Text categorization helps in assigning keywords or labels to documents, making them easy to

search and retrieve. It is widely used in libraries, research systems, and digital databases for efficient information access.

2. Automated Document Organization

It automatically organizes large collections of documents such as corporate records, research papers, and news articles into meaningful categories. This reduces manual effort and improves data management.

3. Content Personalization and Filtering

Text categorization is used to provide personalized content to users based on their interests. It is applied in systems like news recommendations, email filtering, and targeted advertisements.

4. Spam Detection

It is widely used in email systems to classify messages as spam or non-spam. This helps users avoid unwanted or harmful content.

5. Word Sense Disambiguation

Text categorization helps in understanding the correct meaning of words based on context. This improves search accuracy and language processing applications.

6. Web Content Categorization

It organizes online content into hierarchical categories such as Technology, Education, and Entertainment. This makes browsing and navigation easier for users.

Overall, text categorization is widely used in real-world applications to improve information organization, reduce manual work, and enhance system performance.

CHAPTER 7
TESTING

7.1 INTRODUCTION

This chapter covers the testing activities performed for the Text Document Categorization System using Machine Learning. Testing is a critical phase because the system is responsible for automatically classifying text documents into appropriate categories using machine learning techniques. If the system produces incorrect predictions or fails to process input documents, it may affect the overall reliability and usefulness of the application.

The objective of testing is to verify that all functionalities of the system work correctly. The system should allow users to upload documents, process the text, and classify it accurately into predefined categories. It must also handle invalid inputs, empty files, and unsupported formats properly.

Testing was designed to cover both:

- Positive scenarios (valid document upload, correct classification, proper display of results)
- Negative scenarios (invalid file format, empty input, incorrect data)

Testing was performed using two tools:

- **Selenium IDE:** Record-and-playback based UI test automation tool used for quick scenario capture and regression checks.
- **Selenium WebDriver:** Scripted browser automation tool used for repeatable, assertion-based test execution with evidence-level results.

7.2 TEST PLAN

The test plan outlines the strategy and activities required to test the Text Document Categorization using Machine Learning. It defines the scope, objectives, testing environment, tools, and conditions for successful testing.

This plan ensures that all major functionalities, such as document upload, text processing, and category prediction, are verified properly. It also focuses on checking system behavior for both valid and invalid inputs to ensure reliability.

The goal of this test plan is to confirm that the system produces accurate classification results and handles errors effectively. Testing is carried out using manual methods along with automation tools like Selenium IDE and Selenium WebDriver for better efficiency and consistency

Test Plan:(Text Document Categorization System using Machine Learning)

Version: 1.0

Test Plan ID: TP TXT_CAT_01

1. Introduction:

This test plan describes the testing process For Text Document categorization system developed using Django and machine learning techniques. The system classifies text document into predefined categories using features extraction and classification algorithms

2. Test Items:

- File upload module
- Text Preprocessing module
- Feature extraction using TF-IDF
- classification using multinomial
- Database Storage Module
- Output display module

3. Features to be Tested:

- Upload ZIP File functionality
- Extraction of txt files
- Text Preprocessing
- Features extraction using TF-IDF Document classification.
- Display of category, keyword and confidence store
- Data Storage in database

4. Feature Not to be Tested:

- PDF/DOC File handling
- External API integration
- Advanced UI design

5. **Approach:**

- Functional and non- functional testing will be performed
- Positive and negative test case will be executed
- Manual testing and selenium automation will be used

6. **Pass/Fail criteria:**

- All high-priority test cases must be pass successfully
- system should not have any critical or blocking defects.
- Expected results should match actual results for test.

Cases:

- All major defects must be fired and retested
- If incorrect output, system crash, or errors occurs

7. **Suspension Criteria & Resumption Requirement:**

Suspension criteria:

- system crash
- critical bug in fie upload or prediction

Resumption:

- After fixing the issue and verifying the solution.

8. **Test Deliverable:**

- Test Plan document
- Test cases
- Defect Reports
- Test Result Reports
- Final Testing summary

9. **Test Tasks:**

- Prepare test cases
- Execute test case
- Record results

- fix bugs

10. Test Environment:

- OS**:** Windows (window 11)
- Language: Python
- Framework: Django
- Libraries: Scikit-learn**,** NumPy**,** Pandas**,** Joblib

11. Responsibilities:

Developer: Develop and fix bugs.

Tester: Perform testing and report defects Review and provide suggestions Guide

12. Staffing and Training:

Team member learned Python, Django and machine learning Basic knowledge of resulting technique required

13. Scheduling:

Project deadline: 14 MAR 2026, 12:00 PM

14. Risks:

- Accuracy depends on dataset
- Limited file format support
- Performance issue for large files

15. Approvals:

Project Guide, Team Members

Test Cases for file upload module

Table 2: Test cases for File Upload Module

Test Case ID	Objective	Input (Step)	Expected Output	Actual Output	Status
TC_001	Verify valid file Upload	ZIP file with .txt File	File uploaded and categorized Correctly	File uploaded and categorized correctly	Pass
TC_002	Check unsup-ported format	ZIP file with .pdf / .docx	Error Message	File not supported	Fail
TC_003	Validate empty file Handling	Empty .txt file	System Should show warning	Category assigned or others	Fail
TC_004	Verify text Categorization	Valid text document	Correct category predicted	Correct category	Pass
TC_005	Check large file Processing	Large .txt file	Process without delay	Slow processing observed	Fail
TC_006	Verify database Storage	Processed document	Data saved in database	Data stored correctly	Pass
TC-007	Check invalid file Upload	Non-ZIP file	Error message displayed	Error message displayed	Pass

7.3 DEFECT REPORT

The defect report is prepared to document the issues identified during the testing of the Text Document Categorization System. It includes detailed information about the problems encountered while executing test cases, especially in areas like file upload, input validation, and document classification.

The purpose of this report is to clearly describe each defect by comparing the expected outcome with the actual system behavior. This helps the development team to analyze the root cause and implement appropriate fixes.

In addition, the defect report supports effective monitoring of issues by assigning severity and priority levels. It ensures that all identified defects are properly tracked, resolved, and verified, thereby improving the overall performance and reliability of the system.

Defect Report 1

Table 3: Defect Report 1

Defect ID	DR001
Project Name	Text Document Categorization System
Release Version	1.0
Detected in built version	1.0
Summary	System does not processes .txt file and does not support format like pdf or DOCX
Description	During testing, the system failed to process files inside the ZIP when they were in .pdf or .docx format. Instead of properly handling or rejecting them with a clear message, the files were ignored or not processed, causing incorrect system behavior.
Steps to Replicate	1. Upload ZIP with .pdf or .docx 2. Click Upload
Expected Result	The system should display an appropriate error message for unsupported file formats or allow only supported formats.
Actual Result	Files are ignored or not processed without proper error handling.

Severity	Medium
Priority	High
Reported By	ABC
Status	Open

How can Resolve?

Defect DR001 – Resolution

The issue can be resolved by implementing proper file validation before processing. The system should allow only supported file formats such as .txt and reject unsupported formats like .pdf or .docx with a clear error message. Additionally, file type checking should be applied both on the frontend and backend to ensure correct handling.

Defect Report 2

Table 4: Defect Report 2

Defect ID	DR002
Project Name	Text Document Categorization System
Release Version	1.0
Detected in built version	1.0
Summary	System does not properly handle empty files.
Description	During testing, it was observed that when an empty .txt file is up loaded, the system processes it without performing proper validation. Instead of displaying a warning message, the system assigns a default category such as “others,” which leads to incorrect classification.
Steps to Replicate	1. Upload empty .txt file 2. Click Upload
Expected Result	The system should display a warning message such as “Empty File” and restrict further processing.
Actual Result	The system assigns the category “others” without displaying any warning
Severity	Low
Priority	Medium
Status	Open

How can Resolve?

Defect DR002 – Resolution

The defect can be fixed by adding a validation check for empty files before processing. The system should verify whether the uploaded .txt file contains any meaningful data. If the file is empty, the system must display a warning message like **"**Empty File**"** and stop further processing. This ensures that invalid inputs are not classified and improves system reliability.

Defect Report 3

Table 5: Defect Report 3

Defect ID	DR003
Project Name	Text Document Categorization System
Release Version	1.0
Detected in built version	1.0
Summary	Incorrect categorization for valid input
Description	During testing, it was observed that the system sometimes predicts an incorrect category even when valid and relevant text input is provided. This issue may occur due to limitations in the training dataset or model accuracy, leading to unreliable classification results.
Steps to Replicate	1. Upload valid .txt file 2. Click category
Expected Result	Correct category
Actual Result	Wrong category predicted
Severity	Medium
Priority	High

Status	Open

How can Resolve?

Defect DR003 – Resolution

This issue can be addressed by improving the performance of the machine learning model. The accuracy can be enhanced by using a larger and more balanced dataset, applying better text preprocessing techniques (such as tokenization and stop-word removal), and retraining the model. Hyperparameter tuning and model evaluation should also be performed regularly to reduce incorrect predictions and improve overall classification accuracy.

7.4 Testing Output:

1. TC_00 Valid File Upload:

```java
    // TC_001 - Valid File Upload
    public static void tc001_ValidFileUpload() throws InterruptedExceptic
        WebDriver driver = setupDriver();
        System.out.println("TC_001 - Valid File Upload");
        uploadFile(driver, "education_article.txt");
        Thread.sleep(1000);
        System.out.println("TC_001 - PASS");
        driver.quit();
    }
```

```
Problems  @ Javadoc  Declaration  Console ×  Progress  Install Java 26 Support
<terminated> FileUploadTestCases [Java Application] C:\Users\DELL\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.10.v20260205-0638\jre\bin\javaw.exe (Apr 1, 2026, 8:53:30 PM – 8:53:44 PM elapsed 0:0
TC_001 - Valid File Upload
File: education_article.txt
TC_001 - PASS
```

2. TC_002 - Unsupported File Format

```java
    // TC_002 - Unsupported File Format
    public static void tc002_UnsupportedFileFormat() throws InterruptedE
        WebDriver driver = setupDriver();
        System.out.println("TC_002 - Unsupported File Format");
        uploadFile(driver, "unsupported_file.xyz");
        Thread.sleep(1000);
        System.out.println("TC_002 - PASS");
        driver.quit();
    }
```

```
Problems  @ Javadoc  Declaration  Console ×  Progress  Install Java 26 Support
<terminated> FileUploadTestCases [Java Application] C:\Users\DELL\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.10.v20260205-0638\jre\bin\javaw.exe (Apr 1, 2026, 8:53:30 PM – 8:53:44 PM elapsed
TC_002 - Unsupported File Format
File: unsupported_file.xyz
TC_002 - PASS
```

3. TC_003 - Empty File Upload

```java
    // TC_003 - Empty File Upload
    public static void tc003_EmptyFileUpload() throws InterruptedExceptic
        WebDriver driver = setupDriver();
        System.out.println("TC_003 - Empty File Upload");
        uploadFile(driver, "empty_file.txt");
        Thread.sleep(1000);
        System.out.println("TC_003 - PASS");
        driver.quit();
    }
```

```
Problems  @ Javadoc  Declaration  Console  Progress  Install Java 26 Support
<terminated> FileUploadTestCases [Java Application] C:\Users\DELL\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.10.v20260205-0638\jre\bin\javaw.exe (Apr 1, 2026, 8:53:30 PM – 8:53:44 PM elapsed 0:00:14
TC_003 - Empty File Upload
File: empty_file.txt
TC_003 - PASS
```

4. TC_004 Large file Upload

```java
    // TC_004 - Large File Upload
    public static void tc004_LargeFileUpload() throws InterruptedExceptic
        WebDriver driver = setupDriver();
        System.out.println("TC_004 - Large File Upload");
        uploadFile(driver, "large_file.txt");
        Thread.sleep(2000);
        System.out.println("TC_004 - PASS");
        driver.quit();
    }
```

```
Problems  @ Javadoc  Declaration  Console  Progress  Install Java 26 Support
<terminated> FileUploadTestCases [Java Application] C:\Users\DELL\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.10.v20260205-0638\jre\bin\javaw.exe (Apr 1, 2026, 8:53:30 PM – 8:53:44 PM elapsed 0:00:14
WARNING: Unable to find an exact match for CDP version 146, returning the clo
TC_004 - Large File Upload
File: large_file.txt
TC_004 - PASS
=== ALL TEST CASES EXECUTED ===
```

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 CONCLUSION:

The project Text Document Categorization using Machine Learning presents an automated news categorization system that organizes news articles into predefined categories such as Politics, Sports, Business, Technology, and Entertainment using the Naive Bayes machine learning algorithm. By applying text preprocessing and feature extraction techniques, the system efficiently classifies large volumes of digital news content in a structured manner The project helps reduce information overload and enables faster access to relevant news content. It improves usability, saves users' time, and enhances overall content management efficiency. The system also provides students with practical experience in machine learning, text mining, and web application development, making it a useful and effective solution for modern digital news platforms.

8.2 FUTURE SCOPE:

The current system provides an efficient solution for categorizing text documents, but there are several areas where it can be further enhanced to improve performance and usability.

1. Use of Advanced Machine Learning Models

In future, more advanced algorithms such as Deep Learning models (LSTM, BERT) can be used to improve classification accuracy and handle complex text data more effectively.

2. Support for Multiple File Formats

The system can be extended to support different file formats such as PDF, Word documents, and images (using OCR), instead of only text files and ZIP folders.

3. Real-Time Processing

Real-time document classification can be implemented so that users can see instant results without waiting for the entire processing pipeline to complete.

4. Multi-Language Support

Future versions can support multiple languages, allowing users to classify documents written in different languages.

5. Improved User Interface

The interface can be further enhanced with better visualization, charts, and dashboards to make the system more interactive and user-friendly.

6. Integration with Cloud Services

The system can be deployed on cloud platforms to handle large-scale data and provide better accessibility and scalability.

7. Enhanced Model Evaluation

Additional performance metrics and visualization tools can be added to give deeper insights into model accuracy and performance.

8. Custom Category Training

Users can be allowed to define and train the system with their own categories based on specific requirements.

CHAPTER 9

REFERENCE AND BIBLIOGRAPHY

9.1 REFERENCE:

9.1.1 Research Papers

- Knight, K. Mining online text. Commun. ACM 1999,42, 58–61. [CrossRef]
- Pazienza, M.T. Information Extraction; Springer: Berlin/Heidelberg, Germany, 1999.
- Sebastiani, F. Text categorization: Advances and challenges. Comput. Linguist. 2024,50, 205–245, P.3.
- Yang, Y.; Joachims, T. Text categorization. Scholarpedia 2008,3, 4242. [CrossRef]
- Lewis, D.D.; Hayes, P.J. Special issue on text categorization. Inf. Retr. J. 1994,2, 307–340.

9.1.2 Website

- Python Software Foundation, *Python Documentation*, https://docs.python.org/3/
- Django Software Foundation, *Django Documentation*, https://docs.djangoproject.com/
- Scikit-learn Developers, *Scikit-learn Documentation*, https://scikit-learn.org/

9.2 BIBLIOGRAPHY

9. 2.1 Books :

- Sebastian Raschka, Python Machine Learning, Packt Publishing, 2019.
- TomM. Mitchell, *Machine Learning*, McGraw-Hill, 1997.
- William S. Vincent, *Djangofor Beginners*, Self-Published, 2022.